Project Brief: The Cloud-Native Smart Calculator

Welcome to your first full-stack engineering project! Coming from a business background, you already understand *why* applications need to solve user problems. Now, you are going to bridge that with your computer science knowledge to learn *how* industry-standard applications are built, deployed, and scaled.

Your mission is to build a **3-tier Cloud-Native Smart Calculator**. This isn't just a simple script; it is a fully functioning web application where users can register, log in, perform calculations, and view their historical data.

To complete this, you will use **Python, PostgreSQL, FastAPI, Streamlit, Docker, GitHub Actions, and Azure.**

Below is your step-by-step roadmap. I have provided the goals, your specific tasks, and guiding tips. **I will not give you the code.** Your job is to research the implementation details, read the documentation, and piece the puzzle together.

Phase 1: System Design & Database Schema

Before we write a single line of code, we need a blueprint. Every solid engineering project starts with understanding how data flows and how it is stored.

- **Your Task:**
  1. Draw a visual architecture diagram showing how the Frontend, Backend API, and Database communicate.
  2. Design a relational database schema (Entity Relationship Diagram) for this application. You need at least two tables: one for Users and one for Calculation History.
- **Guiding Questions:**
  - How will you uniquely identify each user?
  - How does a specific calculation record know which user it belongs to? (Hint: Think about Foreign Keys).
  - What exact data types will you use for the numbers being calculated and the timestamps?
- **Pro-Tips:**
  - Never store passwords in plain text. When designing your user table, plan for a password_hash column, not a password column.
  - Use a free tool like draw.io or Excalidraw to sketch your architecture.

Phase 2: The Backend Engine & OOP (FastAPI)

Now you will build the "brain" of the application. The backend handles the business logic, talks to the database, and serves data to the frontend via an API.

- **Your Task:**
  1. Set up a local PostgreSQL database.
  2. Create a FastAPI application in Python.
  3. Create endpoints (URLs) for: User Registration, User Login, Performing a Calculation, and Fetching History.
  4. Write the actual calculator logic using **Object-Oriented Programming (OOP)**. Create a Calculator class that handles the math and catches errors.
- **Guiding Questions:**
  - If a user tries to divide by zero, what should your API return? It shouldn't crash the server; it should return a specific HTTP status code.
  - How will you secure the API so that only logged-in users can fetch their history? (Hint: Research JWT - JSON Web Tokens).
- **Pro-Tips:**
  - Keep your code clean. Do not put everything in one file. Have separate files for your database connection, your API routes, and your calculator logic.
  - Research SQLAlchemy or psycopg2 for connecting Python to PostgreSQL.
  - Research passlib for hashing user passwords securely.

Phase 3: The Frontend Interface (Streamlit)

Next, you will build the user interface. We are using Streamlit because it is a native Python tool heavily used in the Data Engineering and Data Science ecosystem.

- **Your Task:**
  1. Create a Streamlit app with two main views: a Login/Registration screen and the Main Calculator screen.
  2. Write Python code within Streamlit to send HTTP requests to the FastAPI backend you built in Phase 2.
  3. Display the user's calculation history in a clean table format.
- **Guiding Questions:**
  - When the FastAPI backend sends back a "success" token after login, how will the Streamlit app "remember" that the user is logged in as they click around the calculator?
- **Pro-Tips:**
  - Research the Python requests library to learn how to make GET and POST requests from Streamlit to FastAPI.
  - Look deeply into Streamlit's st.session_state documentation. This is critical for managing user logins and remembering state between button clicks.

Phase 4: Containerization (Docker)

"It works on my machine" is a phrase we avoid in software engineering. You will now

containerize your application so it runs exactly the same way anywhere in the world.

- **Your Task:**
  1. Write a Dockerfile for your FastAPI backend.
  2. Write a separate Dockerfile for your Streamlit frontend.
  3. Write a docker-compose.yml file that orchestrates three services: the database, the backend, and the frontend.
- **Guiding Questions:**
  - When everything is running in Docker, your backend can no longer use localhost to find the database. How do containers communicate with each other on a Docker network?
  - What happens if the backend API starts up faster than the PostgreSQL database? How do you prevent the backend from crashing immediately?
- **Pro-Tips:**
  - Your Dockerfiles should use official, lightweight Python base images (like python:3.10-slim).
  - Research the depends_on instruction in Docker Compose.

Phase 5: CI/CD & Cloud Deployment (GitHub & Azure)

Finally, you will deploy this application to the cloud and automate the deployment process using DevOps best practices.

- **Your Task:**
  1. Push all your code to a GitHub repository.
  2. Set up GitHub Actions to automatically trigger whenever you push new code to the main branch. This workflow should build your Docker images.
  3. Deploy your containers to Microsoft Azure so that anyone with an internet connection can use your calculator.
- **Guiding Questions:**
  - Where will you store your database credentials (like the database password) safely so they aren't exposed publicly on GitHub?
- **Pro-Tips:**
  - Look into **Azure App Service** (specifically Web App for Containers) or **Azure Container Apps**. Both are excellent for deploying Dockerized applications.
  - Research "GitHub Secrets" to understand how to securely pass environment variables to your cloud provider.